CausalFlow: Causal Attribution and Counterfactual Repair for Diagnosing LLM Agent Failures

Anonymous Authors[1]

Abstract

Large language model (LLM) agents frequently fail on multi-step tasks involving reasoning, tool use, and environment interaction. While such failures are typically logged or retried heuristically, they contain structured signals about where execution broke down. We introduce CausalFlow, an interventional framework that converts failed agent traces into minimal counterfactual repairs and reusable supervision. CausalFlow models execution traces as directed acyclic graphs induced by explicit step dependencies and computes Causal Responsibility Scores (CRS) via step-level counterfactual intervention to identify failure-inducing steps. For these steps, we generate minimally edited repairs that flip the final outcome to success, producing validated contrastive pairs of the form (wrong step, corrected step). CausalFlow supports two complementary uses: targeted test-time repair that recovers from failures with minimal behavioral drift, and training-time supervision suitable for offline preference optimization or reward modeling. Across four benchmarks spanning mathematical reasoning, code generation, question answering, and medical browsing, CausalFlow converts 42.7% of failed executions into validated minimal repairs with high minimality and causal-consensus scores, and yields measurable improvements in downstream task success without modifying model parameters. These results demonstrate that interventional analysis over structured execution traces provides a principled and scalable mechanism for transforming agent failures into reliability gains and learning-ready supervision.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Large language model (LLM) agents execute complex multi-step procedures involving reasoning, tool invocation, and environment interaction. When such executions fail, supervision is typically restricted to outcome-level feedback (e.g., final answer correctness), leaving ambiguous which intermediate decision caused the failure. This ambiguity limits both reliability and learning: without principled credit assignment over execution traces, failures cannot be systematically repaired or converted into structured supervision.

Prior approaches address agent failures through heuristic retries, critique-driven rewriting, or full solution regeneration. While effective in some cases, these strategies do not explicitly test causal responsibility within structured traces. As a result, they neither isolate the failure-inducing step nor guarantee that repairs correspond to minimal counterfactual corrections.

We introduce CausalFlow, a framework for interventional analysis and counterfactual repair of multi-step agent executions. CausalFlow models an execution trace as a directed acyclic graph induced by explicit step dependencies. Given a failed trace, it performs step-level counterfactual interventions: replacing a candidate step and re-executing downstream computation to test whether the final outcome changes. This defines a Causal Responsibility Score (CRS), identifying steps whose intervention flips failure to success.

For causally responsible steps, CausalFlow generates minimally edited counterfactual repairs and validates them via deterministic re-execution or outcome prediction. Each validated repair yields a structured contrastive pair (wrong step, corrected step), transforming execution failures into reusable supervision (Figure 1). This enables two complementary uses: deploy-time repair that improves reliability without parameter updates, and training-time supervision suitable for

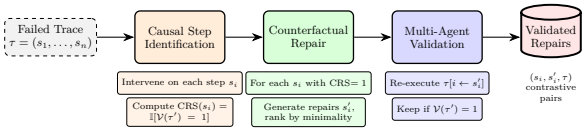

Figure 1. Overview of the CausalFlow pipeline for causal attribution and counterfactual repair. Given a failed execution trace, we identify causally responsible steps via interventional scoring (CRS), generate minimal counterfactual repairs, and validate repairs through re-execution or outcome prediction.

offline preference optimization or reward modeling.

We evaluate CausalFlow across four benchmarks spanning mathematical reasoning (GSM8K), code generation (MBPP), question answering (SealQA Hard), and medical browsing (MedBrowseComp), totaling over 3,000 problems. CausalFlow converts 42.7% of failed executions into validated minimal repairs and demonstrates measurable improvements in downstream task success through targeted counterfactual repair.

Our contributions are:

- A principled interventional framework for step-level causal attribution in structured agent traces.
- A minimal counterfactual repair mechanism that converts failures into validated supervision pairs.
- Empirical evaluation across diverse domains demonstrating scalable supervision extraction and deploy-time reliability gains.
- A formulation enabling offline learning from logged execution trajectories.

2. Related Work

A large body of recent work improves multi-step LLM agents through iterative refinement, self-reflection, and tool-assisted repair mechanisms. Reflexion introduces verbal reinforcement learning where agents critique prior outputs and store reflective feedback for future attempts (?). Self-Refine proposes iterative generation with model-produced feedback to improve responses without additional supervision (?). ReAct interleaves reasoning traces with tool calls to enable interactive problem solving (?), while Tree-of-Thoughts expands reasoning through structured search over intermediate steps (?). CRITIC incorporates external tools to validate and revise model outputs (?), and recent self-debugging approaches enable models to iteratively repair generated code using execution feedback (?). Agentic systems such as SWE-agent further scale iterative debugging to large software repositories (?). Although these methods improve reliability through critique loops, search, or execution feedback, they generally rely on heuristic refinement or full regeneration rather than explicitly identifying and validating the causally responsible step via counterfactual intervention. In parallel, a growing literature demonstrates that supervising intermediate reasoning steps can significantly outperform outcome-only training. Scratchpad supervision encourages models to externalize intermediate computations (?), while process-based reward modeling shows that rewarding correct reasoning steps yields stronger performance and improved interpretability compared to final-answer supervision (?). Large-scale datasets such as PRM800K provide human-labeled step-level reasoning traces to train process reward models (?), and subsequent work scales automated process verifiers and integrates process rewards with structured search and reinforcement-style training (??). However, these approaches rely on curated annotations or learned verifiers rather than extracting supervision directly from naturally occurring failures. Alignment methods such as RLHF train reward models from human preference comparisons and optimize policies using reinforcement learning (?), and recent preference optimization frameworks such as Direct Preference Optimization (DPO) (?), Ranking-based Reward Learning (RRHF) (?), Odds-Ratio Preference Optimization (ORPO) (?), and offline RL approaches such as Implicit Language Q-Learning (ILQL) (?) aim to improve credit assignment and stability without full RL pipelines. Nonetheless, preference signals are typically defined at the trajectory or response level, leaving ambiguity about which intermediate step caused failure. Finally, causal intervention techniques have been applied to analyze and edit internal model behavior, including causal tracing and targeted model editing in transformers (?), investigations into the extent of causal reasoning capabilities in LLMs (??), and formal frameworks for causal abstraction in neural networks (?). CausalFlow differs from these prior directions by operationalizing interventional analysis over explicit execution traces: it models dependency-aware step graphs, performs counterfactual replacement with downstream re-execution, and validates minimal repairs via outcome flips, thereby transforming failed executions into structured contrastive supervision without requiring human step annotations or heuristic full-trace regeneration.

3. Problem Setup

We consider multi-step LLM agents that solve tasks by producing structured execution traces involving reasoning steps, tool invocations, and environment obser-

vations.

Execution traces. For a task instance x, an agent produces a trace

$$\tau = (s_1, s_2, \ldots, s_T),$$

where each step s_t contains an action (e.g., reasoning token sequence or tool call) and, when applicable, the resulting observation returned by the environment. Let $y(\tau)$ denote the task output induced by τ, and let

$$\mathcal{V}(y(\tau), x) \in \{0, 1\}$$

be a task-specific verifier indicating success or failure.

Dependency structure. Execution traces exhibit explicit step dependencies. A step may consume outputs produced by earlier steps, such as intermediate variables, tool responses, or retrieved information. We represent these dependencies as a directed acyclic graph (DAG)

$$G_\tau = (V, E),$$

where $V = \{1, \ldots, T\}$ indexes trace steps and $(j \to t) \in E$ indicates that step s_t depends on information produced at step s_j. Let $\mathrm{Pa}(t)$ denote the parent set of step t in G_τ.

In our implementation, dependencies are logged explicitly from the agent runtime. Tool responses depend on the corresponding tool call; reasoning steps depend on the most recent reasoning step and any tool responses or environment observations they reference; environment observations depend on the immediately preceding environment action. This yields a sparse DAG that reflects operational information flow and enables graph-consistent re-execution without regenerating unaffected steps.

Counterfactual intervention. Given a failed trace τ with $\mathcal{V}(y(\tau), x) = 0$, we define a step-level intervention at index t by replacing s_t with an alternative step $\tilde{s}_t$ and re-executing all downstream steps consistent with the dependency graph. We denote the resulting trace by

$$\tau[t \leftarrow \tilde{s}_t].$$

An intervention is said to be successful if

$$\mathcal{V}(y(\tau[t \leftarrow \tilde{s}_t]), x) = 1.$$

Our goal is two-fold: (1) identify steps whose intervention flips failure to success, and (2) generate minimal counterfactual repairs that yield validated success under re-execution.

4. The CausalFlow Framework

We present the four components of CausalFlow: trace modeling, causal attribution via interventional scoring, counterfactual repair, and multi-agent validation.

4.1. Causal Attribution via Interventions

Given a failed execution trace $\tau = (s_1, \ldots, s_n)$ with $\mathcal{V}(y(\tau), x) = 0$, our goal is to identify which steps are causally responsible for the failure. Inspired by interventionist accounts of actual causation (?), we treat a step as causally responsible if replacing it and propagating its effects forward changes the final outcome.

Graph-aware intervention. Let $G = (V, E)$ denote the causal DAG induced by step dependencies. For a candidate step s_i, we define intervention by:

1. Replace s_i with an alternative s_i'.
2. Remove all descendants $\mathrm{Desc}(s_i)$ in G.
3. Recompute the affected subgraph in topological order.

Steps not in $\mathrm{Desc}(s_i)$ remain unchanged. This ensures interventions respect the explicit dependency structure of the trace rather than performing full regeneration.

Definition 4.1 (Causal Responsibility Score). For a failed trace τ, we generate K intervention proposals $\{s_i'^{(k)}\}_{k=1}^{K}$ for step s_i. The Causal Responsibility Score is defined as:

$$\mathrm{CRS}(s_i) = \max_{k \in \{1,\ldots,K\}} \mathbb{I}\Big[\mathcal{V}\big(y(\tau[i \leftarrow s_i'^{(k)}]), x\big) = 1\Big], \tag{1}$$

where $\tau[i \leftarrow s_i'^{(k)}]$ denotes DAG-consistent re-execution after intervening on s_i and recomputing only affected descendants.

Intuitively, $\mathrm{CRS}(s_i) = 1$ if at least one replacement of s_i flips the verifier outcome from failure to success under DAG-consistent re-execution.

Computing Interventions. For each step s_i, we prompt an LLM with the step content, its dependency context, and failure feedback to generate K minimally edited correction proposals. For reasoning steps, this corrects logical errors; for tool calls, this adjusts arguments or tool selection.

Re-execution. Intervened traces are evaluated through:

Algorithm 1 Causal Responsibility Scoring

Input: Failed trace $\tau = (s_1, \ldots, s_n)$, instance x, verifier $\mathcal{V}(\cdot, x)$
Output: CRS scores $\{\mathrm{CRS}(s_i)\}_{i=1}^{n-1}$
for $i = 1$ to $n-1$ do
  $\mathrm{CRS}(s_i) \leftarrow 0$
  $\{s_i'^{(k)}\}_{k=1}^{K} \leftarrow \mathrm{GenerateInterventions}(s_i, \tau_{\mathrm{Pa}(i)}, \mathrm{feedback})$
  for $k = 1$ to K do
    $\tau' \leftarrow \mathrm{GraphAwareReExec}(\tau, i, s_i'^{(k)})$
    if $\mathcal{V}(y(\tau'), x) = 1$ then
      $\mathrm{CRS}(s_i) \leftarrow 1$
      break
    end if
  end for
end for
return $\{\mathrm{CRS}(s_i)\}$

- Deterministic re-execution: When a domain executor exists (e.g., Python interpreter), affected steps are recomputed exactly.
- Predictive re-execution: Otherwise, an LLM estimates whether the intervened trace would succeed.

Algorithm 1 summarizes the CRS computation.

4.2. Counterfactual Repair

For steps with $\mathrm{CRS}(s_i) = 1$, we generate validated counterfactual repairs.

Minimality. We encourage minimal interventions to preserve interpretability and isolate the root cause. Minimality is measured via token-level Jaccard similarity:

$$\mathrm{Minimality}(s_i, s_i') = \frac{|\mathrm{tokens}(s_i) \cap \mathrm{tokens}(s_i')|}{|\mathrm{tokens}(s_i) \cup \mathrm{tokens}(s_i')|}. \quad (2)$$

Higher values indicate smaller edits.

Repair Selection. Among successful interventions, we select the repair maximizing minimality:

$$\begin{aligned} s_i^\star = \arg\max_{s_i'} \ & \mathrm{Minimality}(s_i, s_i') \\ \text{s.t.} \quad & \mathcal{V}\big(y(\tau[i \leftarrow s_i']), x\big) = 1 \end{aligned} \quad (3)$$

Each validated repair yields a contrastive supervision pair $(s_i, s_i^\star)$.

4.3. Multi-Agent Validation

LLM-generated attributions may be noisy. To improve reliability, we introduce a three-agent validation system:

- Agent A (Proposer): Computes CRS and proposes causal steps.
- Agent B (Critic): Reviews and critiques proposed causal attributions.
- Agent C (Meta-Critic): Reviews both prior judgments and provides a final assessment.

Each critic agent outputs:

- Agreement: {AGREE, PARTIAL, DISAGREE}
- Confidence: $[0, 1]$

We define a consensus score:

$$\mathrm{Consensus}(s_i) = \frac{1}{3}\Big(\mathrm{CRS}(s_i) + \sum_{j \in \{B,C\}} c_j a_j\Big). \quad (4)$$

We map critic judgments to $a_j \in \{0, 0.5, 1\}$ via DISAGREE$\mapsto$ 0, PARTIAL$\mapsto$ 0.5, and AGREE$\mapsto$ 1. Each critic also provides a confidence score $c_j \in [0, 1]$. The normalization factor $1/3$ ensures $\mathrm{Consensus}(s_i) \in [0, 1]$ since $\mathrm{CRS}(s_i) \in \{0, 1\}$. Unless otherwise specified, we set the consensus threshold $\tau_c = 0.5$.

Steps with $\mathrm{Consensus}(s_i) \geq \tau_c$ are retained as confirmed causal steps. This ensemble-based filtering reduces false positives arising from stochastic attribution errors.

5. Experiments

We evaluate CausalFlow across multiple domains to assess its ability to (i) identify causally responsible steps in failed execution traces and (ii) generate validated counterfactual repairs that improve task performance.

5.1. Evaluation Objectives

Our empirical study is designed to answer four primary questions. First, how frequently can CausalFlow convert failed traces into successful executions through targeted intervention? Second, how precise are CRS-based causal attributions, particularly after multi-agent validation? Third, to what extent are generated repairs minimal and localized to the identified causal step? Finally, does deploy-time counterfactual repair measurably improve overall task accuracy compared to the baseline agent?

5.2. Benchmarks

We conduct experiments on four benchmarks spanning mathematical reasoning, program synthesis, web-based question answering, and medical browsing tasks.

GSM8K (?) consists of grade-school math word problems requiring multi-step arithmetic reasoning. MBPP (?) evaluates Python program synthesis with executable correctness checks. SealQA Hard (?) involves complex question answering requiring iterative web search and information synthesis. MedBrowseComp (?) evaluates medical question answering in a browsing environment where the agent must navigate external resources.

Together, these datasets comprise over 3,000 task instances and vary substantially in tool usage, reasoning depth, and environmental interaction complexity.

5.3. Agent Configuration

For each benchmark, we deploy a task-appropriate LLM agent with access to relevant tools. GSM8K is solved using Gemini 2.0 augmented with a calculator tool. MBPP uses GPT-5 Chat with a Python execution environment for deterministic evaluation. SealQA Hard and MedBrowseComp are executed using Gemini 3 Flash with web search and browsing capabilities, respectively.

All agents produce structured execution traces with explicitly logged step types and dependencies, enabling construction of causal graphs for intervention analysis.

5.4. Intervention Protocol

For each failed execution, we compute Causal Responsibility Scores by generating K intervention proposals per step using temperature-based sampling. Unless otherwise specified, we set $K = 3$. Intervened traces are evaluated through deterministic re-execution when a domain-specific executor is available, and through predictive outcome modeling otherwise. Multi-agent critique is applied to filter spurious causal attributions.

For multi-agent validation, we retain causal steps with $\text{Consensus}(s_i) \geq \tau_c$ and use $\tau_c = 0.5$ unless otherwise noted.

5.5. Evaluation Metrics

We report the following metrics. Repair Rate measures the fraction of failed traces that are successfully converted to correct executions. Post-Repair Accuracy measures overall task accuracy after deploy-time repair is applied. CRS Precision evaluates the reliability of identified causal steps. Minimality Score quantifies token-level overlap between original and repaired steps. Finally, Consensus Rate captures agreement among multi-agent validators.

Table 1. Repair performance across benchmarks. Repair Rate is computed over failed traces only. Minimality measures token-level overlap between original and repaired steps (higher = smaller edits).

Benchmark	Total	Passed	Failed	Repairs	Min.
GSM8K	1319	989	330	173 (52.4%)	0.87
MBPP	947	523	488	201 (41.2%)	0.82
SealQA Hard	254	108	146	32 (21.9%)	0.79
MedBrowseComp	484	149	335	149 (44.5%)	0.84

6. Results

We now present empirical results evaluating repair effectiveness, overall performance impact, attribution precision, repair characteristics, and cross-domain behavior patterns across benchmarks.

6.1. Main Repair Performance (RQ1)

Table 1 summarizes repair performance across all domains. CausalFlow successfully converts between 21.9% and 52.4% of failed traces into validated successful executions, demonstrating that a substantial fraction of agent failures are attributable to localized, causally responsible steps. Aggregated over all benchmarks, CausalFlow produces validated repairs for 555 out of 1299 failed executions (42.7%).

GSM8K achieves the highest repair rate (52.4%), indicating that over half of arithmetic reasoning failures can be corrected through localized intervention. Inspection of repaired traces reveals that many failures arise from incorrect intermediate calculations, missed subtraction steps, unit conversion mistakes, or failure to propagate prior results correctly. These errors are typically confined to a single reasoning or calculator step, making them highly amenable to causal intervention.

MBPP exhibits a repair rate of 41.2%. Successful repairs frequently correct boundary conditions, missing base cases, incorrect loop indices, improper conditional checks, and off-by-one errors. Notably, many failures in MBPP are not due to flawed overall program structure, but rather a single incorrect logical decision within otherwise valid code. This aligns with the hypothesis that many LLM-generated programs fail due to localized logic mistakes rather than complete algorithmic misunderstanding.

Table 2. Baseline and post-repair accuracy across benchmarks.

Benchmark	Baseline Accuracy	Post-Repair Accuracy
GSM8K	75.0%	88.1%
MBPP	55.2%	76.4%
SealQA Hard	42.5%	55.1%
MedBrowseComp	30.8%	61.6%

SealQA Hard shows a lower repair rate (21.9%), reflecting the structural difficulty of web-based question answering. A substantial portion of failures stem from retrieval gaps or incomplete information gathering rather than flawed reasoning per se. When missing information cannot be retrieved through modification of internal reasoning steps, local intervention cannot flip the final outcome. Nevertheless, nearly one-quarter of failures remain repairable, typically through improved query formulation or refinement of synthesis reasoning.

MedBrowseComp achieves a repair rate of 44.5%, indicating that many browsing-based medical reasoning failures are localized and correctable. Successful repairs often involve refining search queries, correcting medical term interpretation, or fixing reasoning transitions between retrieved information and final conclusions.

Across all benchmarks, these results suggest that many agent failures are not systemic breakdowns of reasoning, but rather localized causal errors within multi-step traces.

6.2. Impact on Overall Accuracy (RQ4)

We next evaluate deploy-time repair by applying CausalFlow to failed traces prior to returning final outputs. Table 2 reports baseline and post-repair accuracy.

Counterfactual repair yields consistent improvements in overall task accuracy across all domains. On GSM8K, accuracy increases from 75.0% to 88.1%, reflecting the correction of arithmetic and constraint-propagation errors. MBPP improves from 55.2% to 76.4%, demonstrating the effectiveness of targeted logic correction in program synthesis tasks.

SealQA Hard improves from 42.5% to 55.1%, indicating that even in retrieval-heavy settings, a meaningful subset of failures arise from fixable reasoning decisions. The most dramatic improvement is observed in MedBrowseComp, where accuracy increases from 30.8% to 61.6%, highlighting the impact of repairing localized reasoning and navigation errors in browsing-based tasks.

Importantly, these gains are achieved without modifying model parameters. All improvements result from structured causal intervention at inference time, demonstrating that many agent failures can be mitigated through targeted trace repair rather than retraining.

6.3. Minimality and Localization of Repairs (RQ3)

We evaluate the extent to which repairs are minimal and localized. Across benchmarks, average minimality scores range from 0.79 to 0.87, indicating that successful repairs typically involve small token-level modifications rather than wholesale rewriting of traces.

Qualitative inspection confirms that most repairs modify only a single reasoning statement, tool argument, or conditional clause. In the majority of repaired traces, only one or two steps receive CRS $= 1$, suggesting that many failures are attributable to isolated decisions within otherwise coherent reasoning trajectories.

This localization property is particularly pronounced in GSM8K and MBPP, where arithmetic or logic errors occur at identifiable intermediate steps. In browsing tasks, repairs often involve minimal modifications to query phrasing or refinement of inference transitions, again supporting the hypothesis of localized failure mechanisms.

These findings demonstrate that CausalFlow isolates root-cause decisions while preserving unaffected reasoning structure, improving interpretability and maintaining behavioral stability.

6.4. Error Pattern and Skill Decomposition

Clustering causally responsible steps reveals interpretable skill deficits across domains.

In GSM8K, failures cluster into arithmetic operations, multi-digit multiplication, unit conversion, percentage calculations, constraint tracking, and equation setup errors. MBPP failures concentrate around loop construction, string manipulation, recursion handling, boundary conditions, type conversion, and input validation.

SealQA Hard primarily involves query formulation errors, incomplete multi-source synthesis, and improper answer extraction. MedBrowseComp failures frequently involve medical terminology interpretation, symptom-condition mapping, navigation decisions, and reasoning over treatment guidelines.

This decomposition suggests that causal attribution

not only identifies failure-inducing steps but also exposes systematic weaknesses in domain-specific skills. Such signals could potentially guide curriculum-based training or targeted fine-tuning in future work.

6.5. Ablation Studies

We conduct ablation experiments to evaluate the contribution of key components.

Increasing the number of intervention samples K improves repair rates up to $K = 3$, after which gains diminish relative to computational cost. This indicates that limited stochastic exploration is sufficient to identify most repairable failures.

Removing multi-agent critique increases false-positive causal attributions and reduces repair precision, particularly in open-domain settings where reasoning ambiguity is higher. The critique mechanism therefore plays a crucial role in stabilizing attribution reliability.

Eliminating minimality ranking results in larger, less interpretable edits, although overall repair rate remains similar. This suggests that minimality primarily enhances localization and interpretability rather than raw repair success probability.

7. Discussion

7.1. Localized Nature of Agent Failures

A central finding across benchmarks is that many agent failures are localized rather than systemic. In a large fraction of failed traces, only one to three steps receive $\mathrm{CRS} = 1$, indicating that a small number of decisions causally determine the incorrect outcome. This pattern is strongest in structured reasoning domains such as GSM8K and MBPP, where arithmetic or logic mistakes occur at identifiable intermediate steps. However, even in browsing-based environments such as MedBrowseComp, a significant portion of failures arise from localized reasoning transitions or query formulation decisions rather than complete reasoning collapse.

This suggests that LLM agents frequently produce largely coherent reasoning trajectories with isolated errors. As a result, full-trace regeneration may be unnecessary in many cases. Structured intervention focused on causally responsible steps allows preservation of valid reasoning structure while correcting only the failure-inducing decisions.

7.2. Causal Intervention as an Operational Tool

Unlike heuristic self-debugging approaches that rewrite reasoning based solely on global feedback, CausalFlow explicitly models step dependencies and performs graph-consistent intervention. Only descendants of an intervened step are recomputed, reducing behavioral drift and preserving unaffected reasoning components.

The empirical gains observed across domains demonstrate that causal attribution is not merely explanatory but operational. Identifying and modifying specific failure-inducing steps leads to measurable improvements in overall task accuracy without modifying model parameters. This indicates that structured causal debugging can serve as a practical reliability mechanism at inference time.

7.3. Domain-Dependent Repairability

Repair effectiveness varies meaningfully across domains. Closed-domain tasks with deterministic evaluators, such as arithmetic reasoning and program synthesis, exhibit higher repair rates. In these settings, failures typically arise from discrete and correctable reasoning or logic errors within available information.

In contrast, retrieval-heavy tasks such as SealQA Hard exhibit lower repair rates. When failures stem from missing or inaccessible external information rather than flawed reasoning, local intervention cannot flip the outcome. This highlights an important boundary condition: causal repair is effective when the failure originates from incorrect internal decisions, but not when the underlying issue is incomplete evidence acquisition.

7.4. Interpretability and Skill Signals

Beyond performance improvement, CausalFlow provides interpretable insights into systematic agent weaknesses. Clustering causally responsible steps reveals domain-specific skill deficits, including arithmetic propagation errors in GSM8K, boundary-condition logic errors in MBPP, query formulation mistakes in SealQA Hard, and medical reasoning misinterpretations in MedBrowseComp.

These patterns suggest that causal attribution exposes structured behavioral weaknesses rather than isolated random errors. Such signals could inform targeted training strategies, curriculum design, or structured reward shaping, enabling more efficient improvement than generic fine-tuning.

7.5. Limitations

Several limitations warrant consideration.

First, intervention quality depends on the LLM's ability to generate meaningful corrective proposals. If proposed interventions fail to approximate valid alternatives, causally responsible steps may be underidentified.

Second, computing CRS requires re-execution of affected trace segments, introducing computational overhead. Although empirical results indicate that small values of K are sufficient, scaling to very long or tool-intensive traces may require approximation strategies.

Third, retrieval-driven failures cannot be corrected through local reasoning modification when essential information is unavailable. Integrating retrieval-aware intervention mechanisms remains an open challenge.

Finally, the quality of causal attribution depends on accurate dependency annotations. Incomplete or noisy dependency graphs may weaken intervention semantics and attribution reliability.

7.6. Future Work

Several promising directions follow from this work.

Validated counterfactual pairs $(s_i, s_i^\star)$ provide step-level supervision that could be used for targeted fine-tuning, reward modeling, or offline preference learning without requiring manual annotation. Structured causal monitoring could also be integrated into online agent execution, enabling self-repair before final output generation.

Extending causal debugging to multi-agent systems may reveal interaction-level failure modes and coordination errors. Additionally, learning to approximate CRS directly without full re-execution could reduce computational cost and enable scalable deployment.

More broadly, combining causal intervention with retrieval optimization or memory augmentation may address failure modes that are currently outside the scope of local reasoning repair.

8. Conclusion

We introduced CausalFlow, a structured framework for causal debugging of LLM agent execution traces. By modeling step dependencies as directed acyclic graphs and performing graph-consistent counterfactual interventions, CausalFlow identifies causally responsible steps and generates validated minimal repairs. Across four diverse benchmarks spanning mathematical reasoning, program synthesis, web-based question answering, and medical browsing, our method converts a substantial fraction of failed traces into successful executions and significantly improves overall task accuracy without modifying model parameters.

Our results suggest that many LLM agent failures are localized rather than systemic. Structured causal intervention enables correction of failure-inducing decisions while preserving coherent reasoning structure, bridging interpretability and operational reliability. Beyond deploy-time repair, the validated counterfactual pairs produced by CausalFlow offer a principled source of step-level supervision for future training paradigms.

As LLM-based agents become increasingly complex and multi-step reasoning becomes standard, understanding not only whether agents fail but why they fail will be critical. We believe causal intervention over structured execution traces provides a promising direction for improving reliability, interpretability, and ultimately trust in autonomous reasoning systems.

Impact Statement

This paper presents CausalFlow, a framework for identifying and repairing causally responsible steps in LLM agent execution traces. The primary goal of this work is to improve the reliability and interpretability of multi-step AI systems.

By enabling structured causal debugging, CausalFlow may contribute to safer deployment of LLM-based agents in domains such as education, programming assistance, information retrieval, and healthcare support. Identifying localized failure causes can facilitate human oversight, targeted correction, and more transparent system behavior.

At the same time, systematic analysis of failure modes could potentially be misused to probe system weaknesses or optimize adversarial inputs. However, the techniques presented here do not introduce new attack capabilities beyond what is already possible through standard evaluation and stress-testing practices. Instead, the framework is intended to support robustness analysis and reliability improvement.

Overall, we believe that advancing methods for structured failure diagnosis and repair promotes safer and more interpretable AI systems. We encourage future work to integrate such causal debugging tools into broader safety and alignment pipelines.

References

A. Implementation Details

This section provides additional details on trace logging, causal graph construction, intervention prompting, and repair validation.

A.1. Trace Logging and Step Typing

We implement a TraceLogger that records agent execution as a sequence of typed steps. Each step contains:

- A discrete step type,
- Step-specific content (e.g., reasoning text, tool arguments, or observations),
- Explicit dependencies on prior steps.

The step types are defined as follows:

```
class StepType(Enum):
    REASONING = "reasoning"
    TOOL_CALL = "tool_call"
    TOOL_RESPONSE = "tool_response"
    LLM_RESPONSE = "llm_response"
    MEMORY_ACCESS = "memory_access"
  ENVIRONMENT_ACTION = "environment_action"
  ENVIRONMENT_OBSERVATION = "environment_observation"
    FINAL_ANSWER = "final_answer"
```

Each step maintains a list of dependency indices referencing earlier steps in the trace. These dependencies define the edges of the causal graph.

A.2. Causal Graph Construction

The causal graph is constructed directly from recorded step dependencies using NetworkX. Nodes correspond to steps and edges represent directed dependency relationships.

```
def _build_graph(self):
    for step in self.trace.steps:
      self.graph.add_node(step.step_id, step_type=step.step_type.value)
    for step in self.trace.steps:
        for dep_id in step.dependencies:
            self.graph.add_edge(dep_id, step.step_id)
```

This construction ensures that intervention and re-execution operate over an explicit directed acyclic graph rather than assuming purely sequential execution.

A.3. Intervention Prompting

To generate candidate counterfactual interventions, we use a structured prompting template. For each candidate step, the LLM receives:

- The trace context up to the intervened step,
- The failed step content,
- Feedback describing the failure outcome.

The prompt template is:

System: You are an expert at debugging agent reasoning steps. Given a step that contributed to a failed execution, suggest a corrected version that would lead to success.

Context: [Previous steps in trace]
Failed Step: [Step content]
Feedback: [Execution failure message]

Generate a minimal correction to this step.

Sampling is performed with temperature-based decoding to generate multiple intervention proposals per step.

A.4. Repair Validation and Re-Execution

Repairs are validated through either deterministic re-execution or predictive outcome estimation, depending on task availability of an executor.

```
def _evaluate_repair_success(self, step_id, repaired_step):
    if self.reexecutor is not None:
        # Deterministic validation
      new_trace = self.reexecutor.execute(repaired_step)
        return new_trace.outcome == "success"
    else:
        # LLM-based prediction
        return self._predict_outcome(repaired_step)
```

For tasks such as MBPP and GSM8K, deterministic execution ensures reliable validation. For browsing-based tasks, outcome prediction relies on LLM-based evaluation of the modified trace.

A.5. GSM8K Experimental Setup

Dataset. We use the GSM8K dataset (?) loaded from HuggingFace (gsm8k, main config, test split), comprising 1,319 grade-school math word problems.

Model. google/gemini-2.0-flash-lite-001 with temperature 0.7.

Agent Configuration. The GSM8K agent (GSM8KAgent) decomposes problems into structured steps: initial reasoning, LLM-generated structured solution with reasoning and calculation steps, calculator tool calls for each operation, and final answer extraction.

Tools. Calculator tool for evaluating mathematical expressions using MathReexecutor.

Prompts. The solve prompt requests step-by-step solutions with: (1) brief reasoning about approach, (2) each calculation step with description, operation type, and exact expression, (3) the final numerical answer.

Validation Method. LLM-based outcome prediction. Gold answers are extracted as numeric values using MathReexecutor.extract_number().

A.6. MBPP Experimental Setup

Dataset. We use the MBPP dataset (?) loaded from HuggingFace (Muennighoff/mbpp), with train/test/validation/prompt splits merged. Total problems: 974.

Model. GPT-5 Chat with temperature 0.2 for code generation.

Agent Configuration. The MBPP agent reuses the HumanEval-style code generator (HumanevalAgent): reasoning step, LLM code generation tool call, generated Python code, Docker code execution tool call, test results, and final pass/fail answer.

Tools. llm_code_generation for generating Python code via LLM, and docker_code_execution for running code in isolated Docker containers with official test cases.

Prompts. Code generation prompt requests complete Python functions with exact function names, preserved signatures/docstrings, required imports, and no extra output.

Validation Method. Deterministic Docker-based execution using HumanevalReexecutor. Tests run in isolated containers; success requires all test assertions to pass.

A.7. SealQA Hard Experimental Setup

Dataset. We use SealQA Hard (?) loaded from HuggingFace (vtllms/sealqa, seal_hard config, test split), comprising 254 complex QA problems requiring web search.

Model. google/gemini-3-flash-preview with temperature 0.0 for solver, 0.3 for agent steps.

Agent Configuration. The BrowseComp agent (BrowseCompAgent) uses a structured tool policy with maximum 15 steps. Actions include: search, open_url, extract, and answer. State tracking maintains gathered facts, search history, and page summaries.

Tools. web_search via Serper API with caching, and web_fetch for fetching and parsing web pages.

Prompts. Agent system prompt instructs the model to search for relevant information, open promising URLs, extract key facts, and provide answers when sufficient information is gathered. LLM-based grader compares extracted final answers against gold answers.

Validation Method. LLM-based grading. Web results are cached for reproducibility.

A.8. MedBrowseComp Experimental Setup

Dataset. We use MedBrowseComp (?) loaded from HuggingFace (AIM-Harvard/MedBrowseComp_CUA), comprising 484 medical QA problems.

Model. google/gemini-3-flash-preview with temperature 0.3 for agent steps.

Agent Configuration. Same as SealQA (BrowseCompAgent) with maximum 10 steps (reduced for medical domain).

Tools. Same as SealQA: web_search, web_fetch.

Validation Method. LLM-based grading using the same grader template as SealQA.

B. Extended Results

B.1. Per-Model Breakdown

Table 3 provides a breakdown of repair performance by base model used for agent execution.

Table 3. Repair performance broken down by base model.

Benchmark	Model	Failed	Repaired	Repair Rate
GSM8K	Gemini 2.0	330	173	52.4%
MBPP	GPT-5 Chat	488	201	41.2%
SealQA Hard	Gemini 3 Flash	146	32	21.9%
MedBrowseComp	Gemini 3 Flash	335	149	44.5%

These results indicate that repair effectiveness is influenced more by task structure than by the specific base model.

B.2. Dataset and Trace Statistics

Table 4 provides detailed statistics for each benchmark, including baseline accuracy and repair outcomes.

Table 4. Dataset and trace statistics. Total / Avg. row shows sums for counts and macro-averages for rates. Fix Rate = Fixed/Failed; Accuracy = Passed/Total.

Benchmark	Total	Passed	Failed	Fixed	Fix Rate	Accuracy
GSM8K	1319	989	330	173	52.4%	75.0%
MBPP	947	523	488	201	41.2%	55.2%
SealQA Hard	254	108	146	32	21.9%	42.5%
MedBrowseComp	484	149	335	149	44.5%	30.8%
Total / Avg.	3004	1769	1299	555	40.0%	50.8%

B.3. Skill Group Details

Causal clustering reveals domain-specific skill categories. The following skill categories organize failure patterns for targeted debugging and potential curriculum learning. Category counts are derived from clustering causal steps using LLM-based skill labeling.

GSM8K Categories (16): Basic arithmetic, multi-digit multiplication, division with remainders, unit conversion, percentage calculation, ratio and proportion, multi-step planning, constraint satisfaction, word problem parsing, variable tracking, comparison operations, time calculations, money calculations, distance/rate/time reasoning, combinatorics basics, and equation setup.

MBPP Categories (12): Task decomposition and planning, algorithmic logic implementation, tool and API integration, code generation and syntactic integrity, requirement interpretation and mapping, data structure and type management, edge case and boundary handling, mathematical and geometric reasoning, regex and pattern synthesis, input parsing and verification, computational efficiency optimization, and unassigned/outlier failures.

SealQA Hard Categories (4): Query precision and formulation, source selection and extraction efficiency, iterative search and refinement, and temporal/contextual grounding.

MedBrowseComp Categories (6): Source relevance and authority assessment, iterative query refinement and narrowing, search query optimization and formulation, information extraction and parsing, redundancy and execution efficiency, and identifier/resource verification.

Using the Taxonomy. These categories enable: (1) identification of systematic skill deficits in agent behavior, (2) targeted debugging of specific failure modes, and (3) potential curriculum-based training strategies that progress from simpler to more complex skills within each domain.

B.4. Causal Attribution Reliability Metrics

In addition to repair success, we report two attribution-reliability metrics that evaluate whether CausalFlow identifies the correct failure-inducing step(s), and how often multi-agent validation agrees with the proposer.

CRS Precision. For each failed trace τ with verifier outcome $\mathcal{V}(y(\tau), x) = 0$, let

$$\mathcal{S}_{\text{pred}}(\tau) = \{i \in \{1, \ldots, n-1\} \ : \ \text{CRS}(s_i) = 1\}$$

be the set of steps flagged as causally responsible by CRS. Since ground-truth causal steps are not directly observed, we operationalize a conservative notion of correctness using repair validation: a predicted causal step is counted as a true positive if there exists at least one intervention proposal for that step that flips the verifier outcome to success under DAG-consistent re-execution. Formally, define

$$\text{TP}(\tau) = \left|\{i \in \mathcal{S}_{\text{pred}}(\tau) \ : \ \exists k, \ \mathcal{V}(y(\tau[i \leftarrow s_i'^{(k)}]), x) = 1\}\right|.$$

Then CRS Precision is

$$\text{Prec}_{\text{CRS}} = \frac{\sum_{\tau \in \mathcal{F}} \text{TP}(\tau)}{\sum_{\tau \in \mathcal{F}} |\mathcal{S}_{\text{pred}}(\tau)|},$$

where $\mathcal{F}$ is the set of failed traces.

Consensus Rate. For each candidate step s_i evaluated in a failed trace, we compute a consensus score $\text{Consensus}(s_i) \in [0, 1]$ from the proposer and critics. We report the fraction of CRS-flagged steps that pass the consensus threshold:

$$\text{ConsRate} = \frac{\sum_{\tau \in \mathcal{F}} |\{i \in \mathcal{S}_{\text{pred}}(\tau) : \text{Consensus}(s_i) \geq \tau_c\}|}{\sum_{\tau \in \mathcal{F}} |\mathcal{S}_{\text{pred}}(\tau)|}.$$

This measures how often the multi-agent validators corroborate the proposer's causal attributions.

Table 5. Attribution-reliability metrics reported in the Appendix. CRS Precision measures how often CRS-flagged steps admit a validated outcome-flipping intervention under DAG-consistent re-execution. Consensus Rate measures the fraction of CRS-flagged steps retained after multi-agent validation.

Benchmark	CRS Precision (↑)	Consensus Rate (↑)
GSM8K	0.72	0.65
MBPP	0.84	0.72
SealQA Hard	0.68	0.55
MedBrowseComp	0.78	0.65